INTEGRATION ARCHITECTURE REFERENCE

Adapter Microservice (AdapterMS)

Dynamic Runtime Model, Route Orchestration & Technical Specification Guide

Runtime: Apache Camel • Spring Integration Framework

Target Platform: Enterprise Core Banking Integration

1. Executive Architectural Overview

Adapter Microservice (AdapterMS) is a configuration-driven integration runtime designed to eliminate repetitive Java development for enterprise system interfaces. Instead of coding specialized connectors, interfaces are expressed declaratively through:

- **Adapter Registrations:** Top-level job metadata, scheduling directives, and route template references.
- **Route Templates:** Discrete, reusable lifecycle stage units containing configuration requirements and route code.
- **Dynamic Camel Spring XML:** Base64-encoded Camel route graphs evaluated and instantiated dynamically at runtime.
- **Runtime Parameter Bindings:** Dynamic endpoint definitions, transformation specifications, and field mapping schemas.

> **Core Runtime Thesis**
> Configuration defines pipeline assembly, Apache Camel supplies execution semantics, and parameter injection tailors runtime behavior to specific enterprise endpoints.

2. Registration Hierarchy & Entity Relationships

The runtime constructs its active execution context directly from hierarchical JSON payloads. The adapter envelope controls the lifecycle and triggers, while contained template blocks supply route definitions.

```
Adapter Definition (Top-Level Job Entity)
    ├── type: SYSTEM | EVENTTOAPI | APITOEVENT | BULKING | DEBULKING | MDAL
    ├── name / id: Logical integration identifier (e.g., "SEPA PAYMENT ADAPTER")
    ├── status: Active state machine status ("START" | "STOP")
    ├── cronExpression / trigger: Polling frequency or execution schedule
    └── templates: List of assigned stage components
            ├── id: Unique endpoint URI identity (e.g., "direct:eventtoapi.json.process")
            ├── type: Execution stage tag (PREPROCESS | PROCESS | POSTPROCESS)
            ├── inputParameters: Mandatory and contextual parameter bindings
            ├── defaultParameters: Fallback parameter configuration
            └── xmlRoute: Base64-encoded Camel Spring XML route definition
```

3. Adapter Pattern Classification & Taxonomy

AdapterMS categorizes integration flows into six distinct architectural models tailored to enterprise banking operations:

Adapter Type	Primary Source	Target Destination	Pipeline Structure	Typical Implementation
DEBULKING	File drop / File storage	HTTP REST API (per record)	PREPROCESS → PROCESS → POSTPROCESS	Fixed-width or delimited file splitting and payment ingestion.

Adapter Type	Primary Source	Target Destination	Pipeline Structure	Typical Implementation
EVENTTOAPI	Kafka topic / JMS queue	External HTTP REST API	Direct PROCESS (or 3-stage)	Real-time CloudEvent ingestion, JOLT transformation, API dispatch.
APITOEVENT	Inbound REST API call	Kafka topic / JMS queue	Inbound API → Transform → Publish	Asynchronous gateway ingestion and message bus broadcasting.
BULKING	Streamed events / DB rows	Consolidated batch file	Collect → Aggregate → File Write	End-of-day settlement batching and clearing extract generation.
SYSTEM	Quartz cron / Poller	Internal microservices	Poller → Execute → Finalize	Periodic cache refresh, reconciliation, and administrative sweeps.
MDAL	Entity modification event	MDAL Data Cache Layer	Payload Ingestion → Cache Sync	Distributed cache synchronization across microservice instances.

4. Three-Stage Lifecycle Engine: Preprocess, Process, Postprocess

To guarantee modularity, complex multi-step interfaces segment their execution path into three discrete lifecycle stages connected via `direct:` Camel endpoints:

Stage 1: PREPROCESS

Handles file pickup, batch integrity validation, metadata extraction, context enrichment, and chunking. Sets up exchange headers and passes control to `process.routeId`.

Stage 2: PROCESS

Executes core business transformations (JOLT/XSLT), extracts positional/JSON fields, resolves path variables, sets HTTP headers, and triggers target APIs or event brokers.

Stage 3: POSTPROCESS

Collects API response records, constructs output audit files, maps response fields according to positional rules, writes result files to storage, and flags job completion.

Direct Route Chaining

Routes coordinate sequentially via internal endpoints: `direct:preprocess → direct:process → direct:postprocess`.

5. Dynamic Parameterization & Exchange Mapping

All definitions in `inputParameters` populate the runtime Camel Exchange (as properties or headers). The standard integration parameters include:

Parameter Name	Target Domain	Functional Description & Usage
`process.routeId`	Orchestration / Flow	Defines the direct entry endpoint for routing between lifecycle templates.
`httpRequestType`	REST API Invocation	Specifies HTTP verb (`POST` , `GET` , `PUT` , `DELETE`).
`httpEndPoint`	Endpoint Resolution	Base HTTP URI; parsed by `HttpEndPointConstructor` for path/query injection.

Parameter Name	Target Domain	Functional Description & Usage
`https.config`	Security / TLS	Boolean flag (`true` / `false`) to activate SSL/TLS context parameters.
`data.transform.spec`	Data Transformation	Base64-encoded JOLT or XSLT specification executed by mediator beans.
`fieldDetails`	Fixed-Length Parsing	Hash-separated field names (e.g. `txnType#debitAcc#amount#creditAcc`).
`fieldPositionDetails`	Fixed-Length Parsing	Hash-separated character offsets (e.g. `0:2#3:8#9:15#16:25`).
`output.folder` / `result.filename`	File Output Delivery	Target file system directory and output file nomenclature for debulked results.

6. Error Classification Taxonomy & Resilience Policy

AdapterMS enforces an explicit error classification contract using the `result.status` exchange header. This categorization governs upstream replay, circuit breaking, and operational troubleshooting:

```
Incoming Exception
  │
  ├─► HttpOperationFailedException (HTTP >= 500)
  │     ├── Redelivery Policy: 1 retry attempt (delay: 200ms)
  │     └── Final Action: Set result.status = RETRIABLE_FAILURE
  │
  ├─► HttpOperationFailedException (HTTP 4xx)
  │     ├── Redelivery Policy: None (immediate abort)
  │     └── Final Action: Set result.status = NON_RETRIABLE_FAILURE
  │
  ├─► java.io.IOException (Socket / Network / Connection Loss)
  │     ├── Action: Hold consumer offset (do not commit Kafka offset)
  │     └── Final Action: Set result.status = SYSTEM_FAILURE
  │
  └─► Generic / Schema Failures (ExpressionEvaluationException, InvalidPayloadException)
        ├── Action: Dispatch exchange to direct:exception.apiEventError.Responder
        └── Final Action: Set result.status = NON_RETRIABLE_FAILURE
```

7. Concrete Integration Blueprint: EVENTTOAPI

Below is a complete, production-grade template definition for mapping real-time CloudEvents to a banking payment REST API.

7.1 Registration Configuration (JSON)

```
{
  "type": "EVENTTOAPI",
  "name": "SEPA PAYMENT EVENT TO REST API CALL",
  "status": "START",
  "templates": [
    {
      "id": "direct:eventtoapi.json.process",
      "type": "PROCESS",
      "inputParameters": [
        { "name": "process.routeId", "value": "direct:eventtoapi.json.process" },
        { "name": "httpRequestType", "value": "POST" },
        { "name": "https.config", "value": "false" },
        { "name": "httpEndPoint", "value": "http://payment-core:3000/order/paymentOrders/
sepaInstantPayment" },
        {
          "name": "data.transform.spec",
          "value":
"WwogIHsgIm9wZXJhdGlvbiI6ICJzaGlmdCIsICJzcGVjIjogeyAiZGViaXRBY2NvdW50SUJBTiI6ICJib2R5LmRlYml0QWNjb3VudElCQU4iLCAic
        }
      ]
    }
  ]
}
```

7.2 Decoded JOLT Transformation Specification

```
[
  {
    "operation": "shift",
    "spec": {
      "debitAccountIBAN": "body.debitAccountIBAN",
      "paymentCurrencyId": "body.paymentCurrencyId",
      "amount": "body.amount",
      "beneficiaryIBAN": "body.beneficiaryIBAN"
    }
  }
]
```

7.3 Decoded Camel Spring XML Route Execution Engine

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:camel="http://camel.apache.org/schema/spring"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://camel.apache.org/schema/spring
                           http://camel.apache.org/schema/spring/camel-spring.xsd">

    <import resource="classpath:/coreGenericConfig/ApplicationGenericBeans.xml" />

    <camelContext xmlns="http://camel.apache.org/schema/spring">
        <!-- 5xx Errors: Retry and mark RETRIABLE_FAILURE -->
        <onException>
            <exception>org.apache.camel.http.base.HttpOperationFailedException</exception>
            <onWhen><simple>${exception?.httpResponseCode} >= 500</simple></onWhen>
            <redeliveryPolicy maximumRedeliveries="1" redeliveryDelay="200" retryAttemptedLogLevel="INFO"/
>
            <handled><constant>true</constant></handled>
            <setHeader name="result.status"><constant>RETRIABLE_FAILURE</constant></setHeader>
        </onException>

        <!-- 4xx Errors: Do not retry; mark NON_RETRIABLE_FAILURE -->
        <onException>
            <exception>org.apache.camel.http.base.HttpOperationFailedException</exception>
            <handled><constant>true</constant></handled>
            <setHeader name="result.status"><constant>NON_RETRIABLE_FAILURE</constant></setHeader>
        </onException>

        <!-- IO / Connectivity Errors: Mark SYSTEM_FAILURE -->
        <onException>
            <exception>java.io.IOException</exception>
            <handled><constant>true</constant></handled>
            <setHeader name="result.status"><constant>SYSTEM_FAILURE</constant></setHeader>
        </onException>

        <!-- Main Execution Pipeline -->
        <route id="direct:eventtoapi.json.process" streamCache="true" autoStartup="true">
            <from uri="direct:eventtoapi.json.process" />
            <choice>
                <when>
                    <simple trim="true">${exchangeProperty.data.transform.spec} != "" && null != $
{exchangeProperty.data.transform.spec}</simple>
                    <to uri="bean:joltTransformer?method=process(${body},${exchange})" />
                </when>
            </choice>
            <setHeader name="CamelHttpMethod"><simple>${exchangeProperty.httpRequestType}</simple></
setHeader>
            <setHeader name="httpEndPoint"><simple>${exchangeProperty.httpEndPoint}</simple></setHeader>
            <process ref="httpEndPointConstructor" />
            <toD uri="${header.updatedUrl}?bridgeEndpoint=true" />
        </route>
    </camelContext>

    <bean id="joltTransformer"
class="com.temenos.microservice.mediator.engine.AdapterJoltBasedTransformer" />
    <bean id="httpEndPointConstructor"
class="com.temenos.microservice.mediator.engine.HttpEndPointConstructor" />
</beans>
```